

TrekEasy

DevOps & AWS ECS on Fargate — Complete Documentation

[Project & DevOps Overview](#) • [Manual Runbook](#)

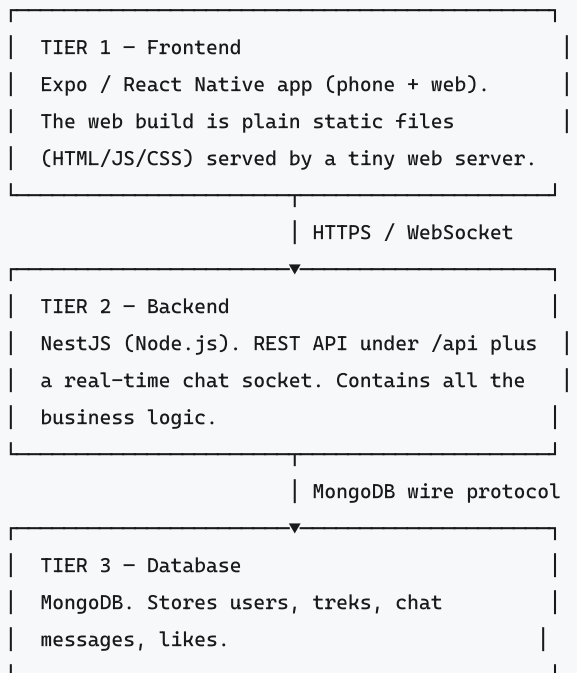
Part 1 — Project & DevOps Overview

TrekEasy — Project & DevOps Overview

A plain-English tour of how TrekEasy is built, containerized, tested, and deployed. No prior DevOps knowledge assumed. If you just want the commands, skip to [RUNBOOK.md](#).

1. What the application is

TrekEasy is a trek-discovery app for the Himalayas: personalized recommendations, trending treks, and a group chat per expedition. It is a **3-tier application**:



"3-tier" simply means the **presentation** (frontend), the **logic** (backend), and the **data** (database) are three separate programs that can be scaled, deployed, and reasoned about independently.

Repository layout

Folder	Tier	What it holds
<code>frontend/</code>	1	Expo app — <code>app/</code> (screens), <code>components/</code> , <code>lib/</code> (API client), <code>context/</code>
<code>backend/</code>	2	NestJS API — <code>src/modules/{auth,users,destinations,chat,recommendations,itinerary,health}</code>
<code>backend-database/</code>	2→3	Mongoose schemas + DB/JWT config. Not a running server — it compiles <i>into</i> the backend image. MongoDB is the actual tier-3 service.
<code>deploy/</code>	—	<code>nginx/</code> reverse-proxy config, <code>apache/</code> alternative
<code>k8s/</code>	—	Kubernetes manifests (Minikube)
<code>aws/</code>	—	ECS on Fargate — CloudFormation + task definitions + scripts
<code>.github/workflows/</code>	—	GitHub Actions CI and CD
<code>Jenkinsfile</code>	—	Equivalent pipeline for a Jenkins server
<code>docker-compose.yml</code>	—	Run all four containers locally with one command

How the frontend finds the backend

The frontend is compiled with `EXO_PUBLIC_API_URL=/api` **baked in** — a *relative* path. When the browser loads the app from `http://some-host/`, it resolves API calls against **that same host** (`http://some-host/api/...`). That is why every deployment (nginx, Kubernetes Ingress, AWS ALB) puts the frontend and backend behind **one hostname** and routes by path. Same origin = no CORS headaches, and the same image works in every environment.

Standardized names

One naming scheme everywhere — Compose, Kubernetes, and AWS:

Tier	Image	Container / task	Network name
frontend	<code>trekeasy-frontend</code>	<code>trekeasy-frontend</code>	<code>frontend</code>
backend	<code>trekeasy-backend</code>	<code>trekeasy-backend</code>	<code>backend</code>
database	<code>trekeasy-database</code>	<code>trekeasy-database</code>	<code>mongo</code>

The database's *network name* stays `mongo` on purpose — it is hard-coded into the connection string `mongodb://mongo:27017/trekeasy` in several places (Compose, `k8s/db-secret.yml`, AWS Cloud Map). Renaming a load-bearing hostname buys nothing; the **image** and **container** names carry the brand.

2. Docker & Dockerfiles

What a container is

A **container** is your app plus exactly the runtime and libraries it needs, packaged into a single read-only bundle called an **image**. The same image runs identically on your laptop, in CI, and on AWS — "works on my machine" stops being a problem. A **Dockerfile** is the recipe for building an image.

The four images

Dockerfile	Base image	Build strategy
<code>backend/Dockerfile</code>	<code>node:20-alpine</code>	multi-stage : stage 1 installs <i>all</i> deps and compiles TypeScript → JS; stage 2 keeps only production deps + the compiled <code>dist/</code> . Smaller, fewer CVEs.
<code>frontend/Dockerfile</code>	<code>node:20-alpine</code> → <code>nginx:alpine</code>	stage 1 runs <code>expo export</code> to produce static web files; stage 2 is just nginx serving that folder on port 8080.
<code>backend-database/Dockerfile</code>	<code>mongo:7-jammy</code>	official MongoDB + our <code>init-indexes.js</code> bootstrap script.
<code>deploy/nginx</code> (compose only)	<code>nginx:alpine</code>	the reverse proxy, config bind-mounted.

Key ideas used here

- **Multi-stage builds** — the compiler and dev dependencies never reach the final image.
- **Build context is the repo root for the backend** — because `backend/tsconfig.json` compiles `backend/` and `backend-database/` together. That is why the compose file and every build command say `docker build -f backend/Dockerfile .` (the `.` is the repo root).
- **Build-time vs run-time config**. `EXPO_PUBLIC_API_URL` is a **build arg** (`--build-arg`) because Expo inlines it into the JS bundle when it exports. Everything the *backend* needs (`MONGODB_URI`, `JWT_SECRET`, ...) is a **runtime env var**, injected when the container starts.
- **HEALTHCHECK** — every image knows how to answer "are you actually working?" (`GET /api/health` for the backend, `GET /` for the frontend, `db.adminCommand('ping')` for Mongo). Compose, Kubernetes, and ECS all use these to decide when a container is ready for traffic.
- **.dockerignore** — keeps `node_modules`, `.env`, and build output out of the build context so images are small and never bake in a secret.

3. NGINX — the reverse proxy

What "reverse proxy" means

A normal proxy sits in front of *clients*. A **reverse proxy** sits in front of *servers*: the outside world talks only to nginx, and nginx decides which internal service should handle each request. Benefits:

- **One public entry point** (one hostname, one port, one TLS certificate).
- **Path-based routing** — `/api` → backend, everything else → frontend.
- **The browser sees a single origin**, so no cross-origin (CORS) problems and cookies "just work".
- A natural place to add caching, rate limiting, gzip, and access logs later.

TrekEasy's routing (`deploy/nginx/nginx.conf`)

Request path	Sent to	Why it is separate
<code>/api/...</code>	<code>backend:3001</code>	the REST API
<code>/socket.io/...</code>	<code>backend:3001</code>	real-time chat; needs the WebSocket <code>Upgrade</code> headers and a long read timeout
<code>/uploads/...</code>	<code>backend:3001</code>	locally-stored images, served <i>outside</i> the <code>/api</code> prefix
<code>/</code> (everything else)	<code>frontend:8080</code>	the Expo web bundle

The exact same four rules appear in `k8s/ingress.yaml` and in the AWS ALB listener rules

(`aws/cloudformation/02-platform.yaml`). nginx, the Kubernetes Ingress, and the ALB are three implementations of one idea: *single front door, route by path*.

4. CI/CD — automated build, test, and deploy

CI (Continuous Integration) = every push is automatically built and tested, so breakage is caught in minutes, not in code review. **CD (Continuous Delivery/Deployment)** = a change that passes CI is automatically packaged and shipped to a running environment.

TrekEasy has **three** pipeline definitions doing the same job for different tools. You only need one; all three are kept working as reference.

4a. GitHub Actions — CI (`.github/workflows/ci.yml`)

Runs on **every push and every pull request**. Two parallel jobs (`backend` , `frontend`), each on Node 18 and 20:

```
checkout → npm ci → lint → typecheck → test → build → upload build artifact
```

If any step fails, the commit is marked **✖** and (for a PR) merging is blocked.

4b. GitHub Actions — CD (`.github/workflows/cd.yml`)

Runs on **push to main** (or manually, choosing `staging` / `production`):

```
test ┌
    │ └─ deploy:
    │   │ 1. assume an AWS IAM role via OIDC (no long-lived keys)
    │   │ 2. docker login to ECR
    │   │ 3. aws/scripts/build-and-push.sh → 3 images to ECR, tagged with the git SHA
    │   │ 4. aws/scripts/deploy.sh → new task-def revisions + rolling update
    │   └─ 5. post the result to Slack (#trekeasy)
```

Deployment secrets/vars live in **GitHub → Settings → Secrets and variables → Actions** — never in the repo. See `RUNBOOK.md §4`.

4c. Jenkins (`Jenkinsfile`)

The same stages expressed as a declarative Jenkins pipeline, for teams running a Jenkins server instead of (or alongside) GitHub Actions:

Stage	What it does
Checkout	pull the commit that triggered the build
Build	<code>npm ci</code> + compile backend and frontend
Test	<code>npm run typecheck</code> + <code>npm test</code> (backend), <code>typecheck</code> (frontend)
Archive	save <code>backend/dist/**</code> and <code>frontend/dist/**</code> as downloadable build artifacts
<i>post</i>	print success/failure, always <code>cleanWs()</code> to free disk

It expects a **Node 20 tool named** `node20` configured in *Jenkins* → *Manage Jenkins* → *Tools*. A production Jenkins setup would add `Push` and `Deploy` stages mirroring the GitHub CD job (build image → push to registry → roll the service).

The pipeline stages, in general terms

Stage	Question it answers	TrekEasy tooling
Build	Does it compile?	<code>npm ci && npm run build</code> , <code>docker build</code>
Test	Does it behave?	<code>jest</code> (88 backend unit tests), <code>tsc --noEmit</code> , <code>eslint</code>
Push	Where's the artifact?	<code>docker push</code> to ECR (<code>trekeasy-backend</code> etc.)
Deploy	How does it reach users?	register a new ECS task-definition revision and <code>update-service</code> → rolling update behind the ALB health check

5. Orchestration manifests — Kubernetes and ECS

Once you have images, something has to **run** them: keep the right number of copies alive, restart crashed ones, roll out new versions without downtime, and wire networking. That "something" is an **orchestrator**. TrekEasy ships configs for two.

5a. Kubernetes (`k8s/`) — for Minikube / any K8s cluster

File	Kind	Plain English
<code>namespace.yaml</code>	Namespace	a private folder (<code>myapp</code>) for all TrekEasy objects
<code>app-config.yaml</code>	ConfigMap	non-secret backend env vars (<code>NODE_ENV</code> , <code>CORS_ORIGINS</code> , ...)
<code>db-secret.yaml</code>	Secret	<code>MONGODB_URI</code> + JWT keys (base64, <i>not</i> encrypted — see the file's note)
<code>mongo-deployment.yaml</code> + <code>-service.yaml</code> + <code>mongo-pvc.yaml</code>	Deployment / Service / PersistentVolumeClaim	MongoDB (1 replica) with a 2 GiB disk that survives restarts
<code>backend-deployment.yaml</code> + <code>-service.yaml</code>	Deployment / Service	the API — 2 replicas , rolling updates, liveness/readiness probes on <code>/api/health</code>
<code>backend-hpa.yaml</code>	HorizontalPodAutoscaler	automatically scale the backend 2 → 6 pods when CPU > 70 %
<code>frontend-deployment.yaml</code> + <code>-service.yaml</code>	Deployment / Service	the Expo web bundle — 2 replicas, ClusterIP (only the Ingress can reach it)
<code>ingress.yaml</code>	Ingress	the single front door on <code>app.local</code> , same path routing as nginx

Core Kubernetes vocabulary: a **Pod** is one running container group; a **Deployment** keeps *N* identical Pods alive and does rolling updates; a **Service** is a stable internal name+IP load-balancing across a Deployment's Pods; an **Ingress** exposes Services to the outside world by hostname/path.

5b. AWS ECS on Fargate (`aws/`) — the target for this project

ECS (Elastic Container Service) is AWS's orchestrator. **Fargate** is the "serverless" mode: you never create or patch EC2 servers — you hand AWS a task definition and it finds capacity to run it. `launchType: FARGATE` throughout.

ECS vocabulary mapped to Kubernetes:

ECS	≈ Kubernetes	TrekEasy
Task definition	Pod spec	one per tier — image, CPU/memory, ports, env, secrets, health check, log config, volumes
Task	Pod	one running copy of a task definition
Service	Deployment	keeps <i>N</i> tasks alive, does rolling updates, registers them with the load balancer
Cluster	(the cluster)	trekeasy-cluster — just a namespace for services
Cloud Map	kube-dns / Service	private DNS: <code>mongo.trekeasy.local</code> → the database task's IP
ALB + target groups	Ingress	single front door, path routing
EFS volume	PersistentVolumeClaim	durable <code>/data/db</code> for Mongo, <code>/uploads</code> for the backend
Secrets Manager	Secret	JWT keys, injected as env vars at task start
Task execution role	—	lets the ECS agent pull images / read secrets / write logs
Task role	ServiceAccount (IRSA)	permissions for the app code itself (empty here — TrekEasy calls no AWS APIs)

The three CloudFormation stacks:

```
01-network.yaml  VPC, 2 Availability Zones, public subnets (ALB + NAT),
                  private subnets (every task runs here, no public IP).

02-platform.yaml 3 ECR repos · 3 CloudWatch log groups · IAM roles ·
                  4 security groups (ALB→app→db→EFS, least privilege) ·
                  internet-facing ALB with the /api,/uploads,/socket.io rules ·
                  encrypted EFS file system + access points ·
                  Cloud Map namespace "trekeasy.local" ·
                  Secrets Manager secret "trekeasy/jwt".

03-services.yaml ECS cluster · backend/frontend/database task definitions
                  (FARGATE) · the 3 services wired to the ALB target groups
                  and Cloud Map.
```

Network security posture: the ALB is the only thing with a public IP. Backend and frontend tasks accept traffic **only from the ALB's security group**; the database accepts traffic **only from the backend's security group**; EFS accepts NFS **only from backend + database**. Outbound internet (for pulling images, Cloudinary, etc.) goes through the NAT Gateway.

6. One change, five environments

You run...	It becomes...
<code>npm run start:dev</code>	local Node process, Mongo on <code>localhost</code>
<code>docker compose up</code>	4 containers on your laptop, nginx on <code>:80</code>
push to a branch	GitHub Actions CI: lint + typecheck + test + build
<code>kubectl apply -f k8s/</code>	2+2+1 pods on Minikube, Ingress on <code>app.local</code>
push to <code>main</code>	GitHub Actions CD → ECR → ECS Fargate rolling update behind the ALB

Same source, same Dockerfiles, same health checks, same routing rules — only the orchestrator changes.

Part 2 — DevOps Runbook

TrekEasy — DevOps Runbook ("What you should do")

Every exact command to set up, build, test, run, and deploy TrekEasy from scratch — locally, on Kubernetes, and on **AWS ECS Fargate** — plus the manual, click-in-the-console steps that **cannot** be scripted.

Commands are **Linux / macOS / WSL / Git-Bash** (`bash`). On native Windows PowerShell, swap `export X=y` for `$env:X="y"` and `$(cmd)` for `$($cmd)` (same) — or just use WSL.

Legend:  = manual action in a browser / console ·  = terminal command.

0. Install the tools (one time, per workstation)



```
# Node.js 20 (via nvm)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
nvm install 20 && nvm use 20
node -v          # v20.x
npm -v

# Docker Engine + Compose plugin (Docker Desktop on Mac/Windows)
docker --version
docker compose version

# AWS CLI v2
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip awscliv2.zip && sudo ./aws/install
aws --version      # aws-cli/2.x

# jq (used by aws/scripts/deploy.sh)
sudo apt-get install -y jq      # or: brew install jq

# Only for the Kubernetes path:
curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install kubectl /usr/local/bin/kubectl
curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

1. Get the code and generate secrets




```
git clone <this-repo-url> TREKEASY-FINAL
cd TREKEASY-FINAL

# Two 64-hex-character JWT signing keys. Keep them somewhere safe.
openssl rand -hex 32      # -> JWT_SECRET
openssl rand -hex 32      # -> JWT_REFRESH_SECRET
```

2. Run it locally — no Docker (fastest inner loop)

You need a MongoDB somewhere. Quickest is a throwaway container:



```
docker run -d --name trekeasy-database -p 27017:27017 mongo:7-jammy
```

Backend



```
cd backend
cp .env.example .env
# edit .env:
#   MONGODB_URI=mongodb://localhost:27017/trekeasy
#   JWT_SECRET=<first key from step 1>
#   JWT_REFRESH_SECRET=<second key>
#   NODE_ENV=development
npm ci
npm run typecheck      # must pass
npm test               # 88 tests, must pass
npm run start:dev      # -> http://localhost:3001/api
```

Verify: 

```
curl -s http://localhost:3001/api/health      # {"status":"ok",...}
```

Frontend (second terminal)



```
cd frontend
npm ci
npm run typecheck
npm run dev      # Expo dev server; press w for web, or scan the QR in Expo Go
```

Useful backend scripts



```
cd backend
npm run seed:destinations      # load the trek catalogue into Mongo
npm run lint
```

3. Run the whole stack locally — Docker Compose



```
cd TREKEASY-FINAL
cp .env.example .env
# edit .env: set JWT_SECRET and JWT_REFRESH_SECRET (from step 1).
# Leave NODE_ENV=production and CORS_ORIGINS=http://localhost as-is.

docker compose up --build      # builds trekeasy-database / -backend / -frontend + nginx
```

Open **http://localhost**.



Day-to-day:

```
docker compose ps              # status + health of the 4 containers
docker compose logs -f backend # tail one service
docker compose down            # stop
docker compose down -v         # stop AND wipe the Mongo volume
docker compose up -d --build backend # rebuild+restart just one service
```

Images produced (named consistently): `trekeasy-frontend:latest`, `trekeasy-backend:latest`, `trekeasy-database:latest`.

4. MANUAL AWS SETUP — do this once, by hand

These steps create identities and permissions. They are deliberately **not** in a script: they need a human with account-owner access and they rarely change.

4.1 AWS account + admin CLI access

1.  Create or use an AWS account. Sign in to the **Console**.
2.  **IAM** → **Users** → **Create user** → name `trekeasy-admin` → attach **AdministratorAccess** (for bootstrapping only; tighten later).
3.  That user → **Security credentials** → **Create access key** → **CLI**.
4.  On your workstation: `bash aws configure` # AWS Access Key ID : <from step 3> # AWS Secret Access Key : <from step 3> # Default region name : us-east-1 (pick one, use it everywhere) # Default output format : json `aws sts get-caller-identity` # confirms you're authenticated

4.2 Choose your region and export it



```
export AWS_REGION=us-east-1
export PROJECT_NAME=trekeasy
```

4.3 Put the JWT keys where the pipeline expects them

`aws/scripts/provision.sh` reads them from your environment and writes them into **AWS Secrets Manager** (`trekeasy/jwt`). Nothing else needs them.



```
export JWT_SECRET=<first key from step 1>
export JWT_REFRESH_SECRET=<second key from step 1>
```

4.4 GitHub → AWS trust, for the CD workflow (skip if deploying only from your laptop)

The CD workflow authenticates to AWS with **OIDC** — no stored AWS keys.

1. **IAM → Identity providers → Add provider** - Type: **OpenID Connect** - Provider URL: `https://token.actions.githubusercontent.com` - Audience: `sts.amazonaws.com`
2. **IAM → Roles → Create role → Web identity** - Identity provider: the one just added - Audience: `sts.amazonaws.com` - Add a trust-policy condition so only *this* repo can assume it: `json "StringLike": { "token.actions.githubusercontent.com:sub": "repo:<ORG>/<REPO>:*" }` - Permissions: start with **AmazonEC2ContainerRegistryPowerUser** + **AmazonECS_FullAccess** + `cloudformation:*` on the `trekeasy-*` stacks (tighten afterwards). Name it `trekeasy-github-deploy`. Copy its ARN.
3. **GitHub repo → Settings → Secrets and variables → Actions - New repository secret:**
`AWS_DEPLOY_ROLE_ARN` = the role ARN - **New repository secret:** `SLACK_WEBHOOK_URL` = your Slack incoming webhook (optional) - **New repository variable:** `AWS_REGION` = `us-east-1`

4.5 (Optional) HTTPS

The templates create an **HTTP :80** listener. For real HTTPS: 1. **Route 53** — register or import a domain. 2. **ACM (Certificate Manager)** in your region — request a public cert for e.g. `trekeasy.example.com`, validate via DNS. 3. **EC2 → Load Balancers → trekeasy-alb → Listeners** — add a **:443** listener with the cert, same rules as :80; change the :80 listener to *redirect to :443*. 4. **Route 53** — A / ALIAS record `trekeasy.example.com` → the ALB. 5. Update the backend's `CORS_ORIGINS` to `https://trekeasy.example.com` (edit `03-services.yaml` or the task definition) and redeploy.

4.6 Environment-variable ownership (who sets what, where)

Variable	Where it lives in AWS	Set by
JWT_SECRET , JWT_REFRESH_SECRET	Secrets Manager <code>trekeasy/jwt</code>	<code>provision.sh</code> from your shell env (§4.3)
MONGODB_URI	backend task definition (<code>mongodb://mongo.trekeasy.local:27017/trekeasy</code>)	CloudFormation, automatic
CORS_ORIGINS , PUBLIC_BASE_URL	backend task definition	CloudFormation, derived from the ALB DNS name
NODE_ENV , PORT , TRUST_PROXY	backend task definition	CloudFormation, static
EXPO_PUBLIC_API_URL	baked into the frontend image at build (<code>/api</code>)	<code>build-and-push.sh --build-arg</code>
CLOUDINARY_*	<i>not set</i> → uploads use the EFS <code>/uploads</code> volume	add to <code>03-services.yaml</code> Environment / Secrets if you adopt Cloudinary
AWS_DEPLOY_ROLE_ARN , SLACK_WEBHOOK_URL , AWS_REGION	GitHub Actions secrets/variables	you, in the GitHub UI (§4.4)

5. Deploy to AWS ECS Fargate — from scratch

Prereqs: §0 (tools), §1 (secrets), §4.1–4.3 done, shell has `AWS_REGION` , `PROJECT_NAME` , `JWT_SECRET` , `JWT_REFRESH_SECRET` exported.

5.1 Create the network + platform (VPC, ALB, ECR, EFS, IAM, secrets)



```
cd TREKEASY-FINAL
chmod +x aws/scripts/*.sh

./aws/scripts/provision.sh          # no tag -> stops after the platform stack
#   deploys:  trekeasy-network    (VPC, subnets, NAT gateway)
#             trekeasy-platform  (ECR repos, ALB, IAM, EFS, Cloud Map, secret)
```

Run with no argument the first time — the ECS **services** stack is created in §5.3, once the ECR repos have images to pull.

5.2 Build and push the three images to ECR



```
./aws/scripts/build-and-push.sh
#   -> <acct>.dkr.ecr.<region>.amazonaws.com/trekeasy-database : <git-sha> + latest
#   -> ...                                           /trekeasy-backend : <git-sha> + latest
#   -> ...                                           /trekeasy-frontend : <git-sha> + latest
```

5.3 Create the ECS services, pinned to a real image tag



```
TAG=$(git rev-parse --short HEAD)
./aws/scripts/provision.sh "$TAG"          # now deploys trekeasy-services, pinned :$TAG

# Wait until all three services are stable:
aws ecs wait services-stable --region "$AWS_REGION" --cluster trekeasy-cluster \
  --services trekeasy-database trekeasy-backend trekeasy-frontend
```

5.4 Get the URL and smoke-test



```
APP_URL=$(aws cloudformation describe-stacks --region "$AWS_REGION" \
  --stack-name trekeasy-platform \
  --query 'Stacks[0].Outputs[?OutputKey==`AppUrl`].OutputValue' --output text)
echo "$APP_URL"

curl -s "$APP_URL/api/health"          # {"status":"ok",...}
curl -sI "$APP_URL/"                  # 200, text/html (the Expo bundle)
```

5.5 Trek catalogue

Nothing to do — the backend seeds the destination catalogue on every boot (`DestinationsService.onModuleInit` , an idempotent upsert). Confirm once the backend is healthy:



```
curl -s "$APP_URL/api/destinations" | head -c 300
```

To force a refresh after editing `backend/src/data/trek-metadata.ts` , redeploy the backend (\$6) — the new task re-runs the upsert.

6. Day-2 operations

Ship a new version (what CD does automatically on push to `main`)



```
TAG=$(git rev-parse --short HEAD)
./aws/scripts/build-and-push.sh "$TAG"
./aws/scripts/deploy.sh "$TAG"          # new task-def revisions + rolling update, waits for stable
```

Roll back



```
aws ecs describe-services --region "$AWS_REGION" --cluster trekeasy-cluster \
  --services trekeasy-backend --query 'services[0].taskDefinition'      # current revision

aws ecs update-service --region "$AWS_REGION" --cluster trekeasy-cluster \
  --service trekeasy-backend --task-definition trekeasy-backend:<PREVIOUS_REVISION>
```

Logs



```
aws logs tail /ecs/trekeasy-backend --region "$AWS_REGION" --follow
aws logs tail /ecs/trekeasy-frontend --region "$AWS_REGION" --since 1h
aws logs tail /ecs/trekeasy-database --region "$AWS_REGION" --since 1h
```

Scale



```
aws ecs update-service --region "$AWS_REGION" --cluster trekeasy-cluster \
  --service trekeasy-backend --desired-count 4
```

(or edit `BackendDesiredCount` in `03-services.yaml` and re-run `provision.sh $TAG`.)

Change a backend env var / secret

- Non-secret: edit `Environment` in `aws/cloudformation/03-services.yaml`, then `./aws/scripts/provision.sh $TAG`.
- Secret:  `bash aws secretsmanager put-secret-value --region "$AWS_REGION" \ --secret-id trekeasy/jwt \ --secret-string '{"JWT_SECRET":"...","JWT_REFRESH_SECRET":"..."}'` `aws ecs update-service --region "$AWS_REGION" --cluster trekeasy-cluster \ --service trekeasy-backend --force-new-deployment`

Open a shell in a running task (debugging)



```
aws ecs update-service --region "$AWS_REGION" --cluster trekeasy-cluster \
  --service trekeasy-backend --enable-execute-command --force-new-deployment
# then:
TASK=$(aws ecs list-tasks --region "$AWS_REGION" --cluster trekeasy-cluster \
  --service-name trekeasy-backend --query 'taskArns[0]' --output text)
aws ecs execute-command --region "$AWS_REGION" --cluster trekeasy-cluster \
  --task "$TASK" --container trekeasy-backend --interactive --command "/bin/sh"
```

Tear it all down (stops billing)



```
aws ecs update-service --region "$AWS_REGION" --cluster trekeasy-cluster --service trekeasy-frontend --desired-count 1
aws ecs update-service --region "$AWS_REGION" --cluster trekeasy-cluster --service trekeasy-backend --desired-count 1
aws ecs update-service --region "$AWS_REGION" --cluster trekeasy-cluster --service trekeasy-database --desired-count 1

aws cloudformation delete-stack --region "$AWS_REGION" --stack-name trekeasy-services
aws cloudformation wait stack-delete-complete --region "$AWS_REGION" --stack-name trekeasy-services
aws cloudformation delete-stack --region "$AWS_REGION" --stack-name trekeasy-platform
aws cloudformation wait stack-delete-complete --region "$AWS_REGION" --stack-name trekeasy-platform
aws cloudformation delete-stack --region "$AWS_REGION" --stack-name trekeasy-network
```

ECR repos and the EFS file system have deletion protection semantics — if a stack delete stalls on them, empty the ECR repos and delete EFS in the console, then retry. The `trekeasy/jwt` secret is retained for 7–30 days by AWS before permanent deletion.

7. Kubernetes (Minikube) — alternative to ECS



```
minikube start
eval $(minikube docker-env)      # build images straight into Minikube's Docker

docker build -f backend/Dockerfile -t trekeasy-backend:latest .
docker build -f frontend/Dockerfile --build-arg EXPO_PUBLIC_API_URL=/api \
    -t trekeasy-frontend:latest frontend
docker build -f backend-database/Dockerfile -t trekeasy-database:latest backend-database

minikube addons enable ingress
minikube addons enable metrics-server

# db-secret.yaml ships with demo JWT keys - replace them first:
# node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
kubectl apply -f k8s/

kubectl get pods -n myapp -w
echo "$(minikube ip) app.local" | sudo tee -a /etc/hosts
curl http://app.local/api/health
```

Rolling update / rollback:

```
kubectl apply -f k8s/backend-deployment.yaml
kubectl rollout status deployment/backend -n myapp
kubectl rollout undo deployment/backend -n myapp
```

8. Full "from zero" checklist

- [] \$0 tools installed (`node -v`, `docker --version`, `aws --version`, `jq --version`)
- [] \$1 repo cloned, two `openssl rand -hex 32` keys saved
- [] Local: `backend npm ci && npm test green; npm run start:dev; /api/health = ok`
- [] Local: `frontend npm ci && npm run typecheck; npm run dev loads`
- [] Compose: `.env` filled; `docker compose up --build`; `http://localhost` works
- [] ⓘ \$4.1 AWS account, `trekeasy-admin` IAM user, `aws configure`, `aws sts get-caller-identity`
- [] 📜 \$4.2–4.3 `AWS_REGION`, `PROJECT_NAME`, `JWT_SECRET`, `JWT_REFRESH_SECRET` exported
- [] ⓘ \$4.4 GitHub OIDC provider + `trekeasy-github-deploy` role + repo secrets/vars (if using CD)
- [] 📜 \$5.1 `./aws/scripts/provision.sh`
- [] 📜 \$5.2 `./aws/scripts/build-and-push.sh`
- [] 📜 \$5.3 `./aws/scripts/provision.sh $(git rev-parse --short HEAD) + ecs wait services-stable`
- [] 📜 \$5.4 `curl $APP_URL/api/health = ok`
- [] 📜 \$5.5 seed task run
- [] ⓘ \$4.5 HTTPS listener + Route 53 record (production)
- [] [] Push to `main` → CD builds, pushes, rolls the services, posts to Slack

9. Troubleshooting

Symptom	Likely cause	Fix
Service won't start, <code>CannotPullContainerError</code>	image tag not in ECR	run <code>build-and-push.sh</code> with that tag first
Backend task cycles, logs show <code>ECONNREFUSED ... 27017</code>	database task not up yet, or <code>mongo.trekeasy.local</code> not resolving	check <code>aws logs tail /ecs/trekeasy-database</code> ; confirm the database service is <code>RUNNING</code> ; Cloud Map record appears ~30 s after the task is healthy
Backend logs: <code>CORS_ORIGINS</code> is not set; refusing all cross-origin	<code>CORS_ORIGINS</code> empty	it is derived from the ALB DNS in <code>03-services.yaml</code> ; re-run <code>provision.sh \$TAG</code>
ALB returns 503	no healthy targets	<code>aws elbv2 describe-target-health --target-group-arn <arn></code> ; check the container health check + security groups
Chat/socket disconnects every ~60 s	idle timeout too low or no stickiness	ALB idle timeout is 300 s and the backend target group has <code>lb_cookie</code> stickiness — confirm the <code>/socket.io/*</code> rule points at the backend TG
<code>expo export</code> OOMs in CI	Node heap	already handled by <code>node:20-alpine</code> ; if it recurs, raise the frontend build's runner size
Mongo data lost on redeploy	task not using the EFS volume	confirm <code>mountPoints /data/db → mongo-data</code> in the database task def; EFS mount targets exist in both AZs
<code>provision.sh</code> fails: <code>JWT_SECRET: unbound variable</code>	secrets not exported	<code>export JWT_SECRET=...</code> <code>JWT_REFRESH_SECRET=...</code> (§4.3)
CD fails at "Configure AWS credentials"	OIDC trust / role ARN	re-check §4.4 trust policy <code>sub</code> matches <code>repo:<ORG>/<REPO>:*</code> and <code>AWS_DEPLOY_ROLE_ARN</code> secret